

Work-In-Progress: Parallel Classification and Dynamic Routing in IDK Classifier Cascades

Abhinav Gupta
College of Engineering
Texas A&M University
College Station, TX 77843, USA
abbigupta.j18@tamu.edu

Albert M. K. Cheng
Department of Computer Science
University of Houston
Houston, TX 77004, USA
cheng@cs.uh.edu

Thomas Carroll
Department of Computer Science
University of Houston
Houston, TX 77004, USA
tcarroll@uh.edu

Abstract—I Don’t Know (IDK) cascades reduce computational cost by using a lightweight model to classify easy inputs while passing more complex inputs to more powerful models. But in a multi-processor scheme, conventional IDK cascades classify inputs sequentially, leaving many resources idle and limiting the number of inputs that can be classified per second for real-time classification. This work introduces parallel classification in an IDK cascade, in which multiple models concurrently process different images. Where established models waste time by having models wait on other models to finish classifying an input we allow a model to begin running a new input as soon as it has finished the previous one. This is reinforced through use of a doubly linked list that saves inputs that were unable to be classified as they traverse through the cascade for execution by a more complex model at a later time. We compare our model against other multi-processing IDK Cascade schemes and show that our model allows for higher classification throughput, an important metric in systems such as autonomous driving at relatively similar classification accuracy.

Index Terms—IDK cascades, Multi-processing, Queues, Doubly Linked list

I. INTRODUCTION

When considering resource use for machine perception problems, it is preferable to use a model with lower computational cost and accurate results than an equally correct model of higher cost. To this end, IDK cascades [1] have been developed to select the lowest cost model while maintaining accuracy. IDK cascades consider a series of models in sequence (models A , B , and C), typically, of increasing accuracy, computation time and complexity. An input begins by running on A and if the confidence of the input on A is lower than a confidence threshold (typically 0.7-0.95 for the whole IDK cascade [2]) then A returns “IDK” and the input is passed to the next model. Normally an IDK cascade runs as a single process where each model is run sequentially on a single processor system. To adapt this to a multi-processor system, previous schemes run all of the models concurrently to classify a single input. These previous methods split the models in the cascade and run a portion of the split models on each processor such that each model is run once [2]. For instance, given models A , B , C , and D and 2 processors, A and D would run on processor 1 while B and C run on processor 2. While this allows for faster processing of the

models within the IDK cascade itself, this limits throughput, and some computation time may be wasted if a cheaper model classifies the input correctly while another more expensive model is also currently running. As we will show this is also the case when considering the same four models and four processors.

Our contribution is as follows: We implement a novel multiprocessor IDK scheme to increase throughput of inputs, we then introduce the concept of using a queue to save inputs between different models. We propose a flexible scheme that runs a model per GPU core with flexible deferral of models between processors in the case that a processor is idle. This allows us to distribute work amongst processors and have a single model constantly classifying simple inputs while other processor handle more difficult inputs, thus increasing throughput.

II. PARALLEL 3 MODEL IDK CASCADE

Our proposed model takes inspiration from previously established IDK cascades [2]–[4] and considers four ResNet models ResNet-18, ResNet-34, ResNet-50, and ResNet-152, although for this paper we consider IDK cascades made up for 3 models, referred to as M_A , M_B , M_C with ResNet-50 and ResNet-152 being used interchangeably as M_C . Our aim is to concurrently process multiple images arriving in sequence across a multiprocessor system using the aforementioned ResNet models in a method similar to IDK cascades. Unlike the previous sequential cascade single and multiprocessing methods, the system does not require one image to complete the entire pipeline before starting to process the next image. Instead, if M_A returns IDK for some input i_x instead of waiting for i_x to be successfully classified by a later model, we allow M_A to begin processing the next input $i_x + 1$. To further elaborate on this, while M_A was processing i_x M_B could have been processing another input i_y and M_C could be processing another input i_z .

This allows for multiple images or frames to be processed concurrently by having each model work on its individual input one at a time. This is done by running each model as its own process and passing the inputs between each other and the doubly linked list as necessary.

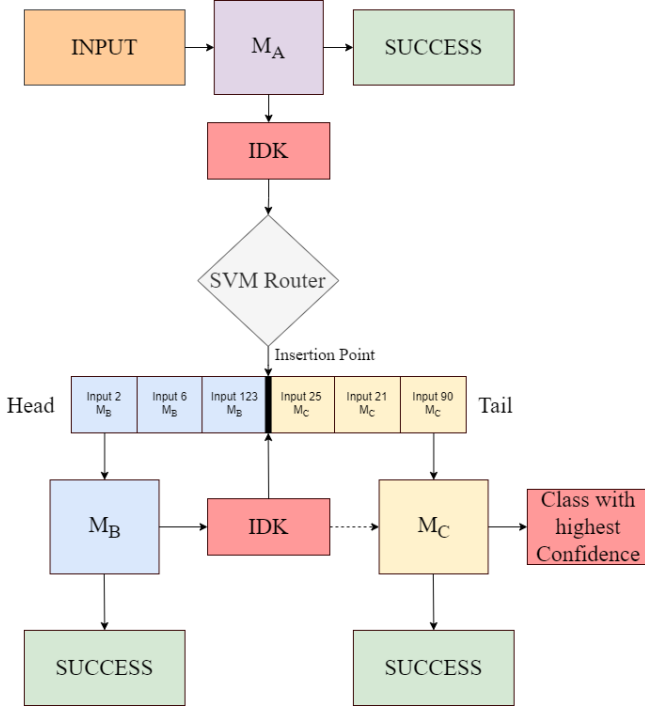


Fig. 1. Parallel 3 Model IDK Cascade

Fig. 1 illustrates our basic 3 ResNet model architecture, every input $i_x \in I$ is run on M_A and, based on the confidence score of M_A i_x is either classified (SUCCESS) or M_A returns “IDK” and the input is sent to a more capable model. If M_A returns “IDK”, before sending i_x to another model, the system extracts three features from its output probability distribution: Confidence, Entropy, and Margin. Where confidence is the confidence of the highest predicted class, entropy is calculated from all probabilities, and the margin is the difference between the top two confidence values, as done in [3].

These features are then passed to a **router** which, in our case, uses a Support Vector Machine (SVM) [5] model to predict whether M_B or M_C is more likely to accurately classify i_x based on our features. We originally tested the SVM against a Random Forest and Extra Tree classifier but chose the SVM because it performed the best during our testing. We trained the router models on the matched-frequency and top-images dataset variants of ImageNet-V2, and used the threshold-0.7 variant for testing.

Returning to the architecture, based on the prediction of the router, i_x is placed into the middle of the doubly linked list at an insertion point pictured in Fig. 1. Which side of the insertion point depends on whether the input has a label to run using M_B or M_C with inputs set to run on M_C on the tails side and inputs set to run on M_B on the head side. M_B and M_C run by selecting the input at the head or tail of the doubly linked list respectively, which we use instead of independent queues to allow for a feature we call work stealing. Job stealing works by allowing M_C to run inputs from the doubly linked that are assigned to M_B if M_C is idle and there are not inputs in the doubly linked list labeled M_C .

This only works one way however, as we do not allow M_B to steal work from M_C as an input being labeled M_C implies that the router predicts that M_B is not able to classify it.

In summary, inputs are initially fed into M_B and M_C via the doubly linked list based on what model they are assigned to be classified by with the exception of the stealing rule. From here if M_B fails to classify an input i_x then i_x is placed back into the doubly linked list on the tail side with the assignment to run on M_C . If an input fails to be classified by M_C then we return the class with the highest classification from M_C which means we may have a high chance of incorrectly classifying the input. While previous techniques have used deterministic classifiers with 100% accuracy and long computation times. We avoid the use of one for now due to our objective of maximizing throughput, and the long overhead potentially “clogging up” the architecture of certain models used during testing. Furthermore, it may not always be the case that a deterministic algorithm is available for these sorts of problems.

To coordinate this system on a multi-processor level, we implement a **central scheduler** that admits new images, knows the real-time status of every model, dispatches uncertain images to the router and then doubly linked list for the appropriate model to classify them while also maintaining references to the head, tail, and insertion point. As seen in Fig. 1, each sample has a unique index, this allows classification to occur in non-sequential order with respect to the order each image entered the system.

The unique index allows us to “save” the state of an input, and potentially return its results out of order from when it entered the system. Compared to a sequential cascade, this approach has the greatest impact on increasing throughput or frames classified each second, as will be shown in section III.

III. EXPERIMENTAL SETUP

Dataset: We use the three 10,000-image ImageNet-V2 test sets [6]. Each contains ten images for each of the 1,000 ImageNet-V2 classes. The *top-images* set contains the candidates with the highest selection frequency, the *threshold-0.7* set samples images selected by at least 70% of annotators, and the *matched-frequency* set matches the selection-frequency distribution of the original ImageNet-V2 validation set. As mentioned we use matched-frequency and top-images to train the router and use threshold-0.7 set for our experiments.

Images are resized to 256 pixels, center-cropped to 224×224 , converted to tensors, and normalized with the ImageNet statistics associated with the pretrained weights. Inference uses a batch size of one for the live experiments.

Hardware and Software: We run our experiments on an Apple M1 MacBook Pro 8 GPU threads via Metal Performance Shaders (MPS) [7] and 16 gigabytes of unified memory, which we use to run the classifiers via PyTorch. Each classifier plus our central scheduler runs as its own Python process or “worker”, but all workers run over the same physical accelerators. It should be noted that each worker is a Python child process responsible for executing one classifier and so a worker is a logical execution context, not a dedicated core.

To give the best possible chance to the implementation from [2] we consider an IDK cascade made up of ResNet-18 $\rightarrow$ ResNet-34 $\rightarrow$ ResNet-50/ResNet-152 (again alternating ResNet-50 and ResNet-152) with each ResNet model running on a worker for a total of 3 workers. With 8 possible MPS GPU threads allowing for 8 concurrent threads present, to ensure all models can run at once. As we will demonstrate however, the method from [2] does not have a mechanism to ensure that the models are constantly running under these conditions. These conditions are not entirely unrealistic for embedded devices either as even lower end embedded devices have multi-core systems.

To show that our model is better at running the models in parallel over previous methods, with regards to throughput, we compare our Parallel three model IDK cascade (P_3) as described in section II and Fig. 1 against two other models each with varying configurations for completeness. We denote the approach from [2] as S_3 , and vary the classification threshold between 0.7 and a threshold OPT designed to optimize the accuracy of the cascade. OPT gives a threshold per model, resulting in lower latency than simply using a fixed threshold as will be seen in our results [8]. As previously mentioned, we also vary the final ResNet model between ResNet-152 and ResNet-50 as ResNet-50's run time allows for faster throughput and allow for a more fair comparison against our model since we use ResNet-50 as our 3rd model.

The second model involves taking a single processor IDK cascade and running three of these cascades across several processors or in our case three workers as to compare with our other three model systems. So a single worker runs an IDK cascade like a normal on a single input i and the next input $i + 1$ is run on the second worker and so on. Using this scheme, each input is considered by a single cascade across multiple workers. We denote this scheme as R_3 as we used an IDK cascade with a Random forest skipping scheme as defined in [3], and like S_3 we vary the final model between ResNet-152 and ResNet-50 and 0.7 and OPT classification thresholds.

Metrics: During our experiments, we measure the accuracy, throughput as FPS and latency. Accuracy is the fraction of all input images whose final classification is correct as stated in the labeled dataset. Throughput is the number of completed inputs divided by the time taken, which in our case since we are interested in FPS, is the number of images that were finished every second. Latency is completion time subtracted from the time an input was admitted into the IDK cascade.

The mean execution time represents the average time needed for a model to process one image. Measurement starts when the worker begins inference and ends when its result is returned. It measures per-model computation time rather than end-to-end latency, which also includes time spent waiting in the queue and going through the scheduler.

We also measure the Final classification count, mean execution time of each model in milliseconds (and the idle time of each model in seconds. The Final classification count is the number of times a model made a classification other than IDK.

The mean execution time is the average time each model took to process all inputs passed to it and the idle time is the time in which the model was not being actively run by the worker.

IV. RESULTS

We report our main metrics in Table I, we compared our 3 models, P_3 , S_3 and R_3 , using the same hardware, software environment, and the Imagenet-v2 threshold-0.7 variant dataset. We label each configuration by the model used (P_3 , S_3 and R_3), the threshold used, (0.7 and optimal OPT mentioned earlier), and the ResNet model used as M_C (ResNet-50 (RN50) and Resnet-152 (RN152)) and number them for each reference. We include the full set of variations for completeness. We can see that variants of our P_3 were able to outperform both R_3 and S_3 in both FPS and latency. In terms of our target metric FPS, we managed to get the best value 58.5 on P_{34} with a range of values between 47.11 – 58.5 with only our lowest FPS of 47.11 on P_{33} being outperformed by R_{32} the best non- P_3 model. The P_3 models had a much broader range of latency values from 15.88 – 61.386 with P_{32} 15.88 being the best recorded across all models and P_{34} the worst at 61.386s. Finally for accuracy we had a range of values from 68.5%–74.78%, with all these values being out done by some version of S_3 at 76.9% and R_3 at 76.1%.

While these metrics are very favorable to our models for our target parameter FPS, this graph requires more context than the metrics that are provided on it.

TABLE I
OVERALL PERFORMANCE COMPARISON OF CASCADE CONFIGURATIONS

Configuration	Classification Accuracy	FPS	Mean latency (ms)
S_{31} T = OPT RN152	69.5%	13.503	17.423
S_{32} T = OPT RN50	69.09%	22.129	21.204
S_{33} T = 0.7 RN152	76.9%	14.09	29.303
S_{34} T = 0.7 RN50	75.79%	26.496	19.963
R_{31} T = OPT RN152	69.5%	44.562	20.416
R_{32} T = OPT RN50	69.1%	49.834	18.094
R_{33} T = 0.7 RN152	76.1%	31.1	30.16
R_{34} T = 0.7 RN50	76.05%	31.848	29.825
(Ours) P_{31} T = OPT RN152	68.5%	58.217	17.412
(Ours) P_{32} T = OPT RN50	68.16%	58.084	15.88
(Ours) P_{33} T = 0.7 RN152	74.78%	47.11	61.386
(Ours) P_{34} T = 0.7 RN50	73.2%	58.5	36.296

To attempt to further explain these results we look to the relevant statistics in table II and try to explain why the P_3 models specifically are better than the R_3 and S_3 models. There are a few generalities that can be attributed to the varying of the threshold value or the switching of M_C . For instance, in almost all cases, models with RN152 have larger latency values than models with RN50 while models using RN50 tend to have higher FPS values. This is reinforced by the latency values attributed to M_C for models that use RN152 in table II-B. Next models that use the OPT threshold scheme tend to have lower latencies at cost of lower accuracy and mostly unchanged FPS. This can be explained by the final classifications in table II-A where models using OPT have more inputs classified by M_A which reduces execution time as seen from the table II-B values from M_A . So essentially more

TABLE II
CLASSIFICATION COUNT, IDLE TIME AND MODEL USAGE CONFIGURATIONS

Final Classification Count (Table II - A)				Mean Execution Time (ms) (Table II - B)				Idle Time (s) By Model (Table II - C)			
Config.	M_A	M_B	M_C	Config.	M_A	M_B	M_C	Config.	M_A	M_B	M_C
S_{31} T=OPT RN152	9186	56	758	S_{31}	12.6	18.617	64.485	S_{31}	614.587	554.413	95.730
S_{32} T=OPT RN50	9186	56	758	S_{32}	18.955	23.405	35.937	S_{32}	262.339	217.841	92.520
S_{33} T=0.7 RN152	5555	1374	3071	S_{33}	12.283	17.890	61.592	S_{33}	586.888	530.826	93.800
S_{34} T=0.7 RN50	5555	1374	3071	S_{34}	13.125	19.987	29.171	S_{34}	246.166	177.548	85.702
R_{31} T=OPT RN152	9186	35	779	R_{31}	11.235	17.187	64.271	R_{31}	112.058	217.513	174.338
R_{32} T=OPT RN50	9186	35	779	R_{32}	11.725	17.816	29.576	R_{32}	83.416	193.524	177.628
R_{33} T=0.7 RN152	5555	986	3459	R_{33}	11.1	17.3	27.0	R_{33}	210.5	281.8	224.7
R_{34} T=0.7 RN50	5555	986	3459	R_{34}	12.707	21.948	25.757	R_{34}	186.921	263.665	224.898
(Ours) P_{31} T=OPT RN152	9186	494	320	(Ours) P_{31}	12.656	22.879	75.003	(Ours) P_{31}	45.211	160.469	147.770
(Ours) P_{32} T=OPT RN50	9186	536	278	(Ours) P_{32}	12.742	22.751	34.698	(Ours) P_{32}	44.741	159.969	162.517
(Ours) P_{33} T=0.7 RN152	5555	2318	2127	(Ours) P_{33}	16.502	20.648	72.025	(Ours) P_{33}	47.252	164.405	59.070
(Ours) P_{34} T=0.7 RN50	5555	2555	1890	(Ours) P_{34}	12.6	18.8	28.1	(Ours) P_{34}	44.7	122.9	117.8

executions of M_A means lower latency and lower accuracy. However what cannot be attributed to M_C or the confidence threshold is that when you vary the fundamental model and look at the equivalent setups such as S_{31} R_{31} and P_{31} or S_{32} R_{32} and P_{32} etc. The P_3 variant always has higher FPS than the other versions.

To explain why this is the case we look at the fundamental differences between P_3 and the other models. While, R_{32} had the highest FPS of any non- P_3 model at 49.834 R_{32} fails to outperform P_{32} due to R_3 fundamentally failing to properly defer tasks like the P_3 . To elaborate if 3 inputs that require M_C or even M_B appear all at once, then rather than running easy inputs on one worker while another worker takes care of the more difficult workers like P_3 all workers would be stuck on a difficult input resulting in decreased FPS. With the values seen in table II-B for R_{32} if an easy input takes 11.725ms and the worst case is $17.816 + 29.576 = 47.392$ ms then that is approximately 4.04 easy inputs that could be processed in the time it would take to classify a single difficult one. This means that the 49.562 FPS reading for R_{31} may match P_3 's FPS based on when difficult inputs appear, but it is likely not to do so consistently. For now we leave analysis of the distribution of R_3 's performance to a later work.

S_3 likely performs worse than the P_3 with regards to FPS for one simple reason, the amount of time the model spends running M_C . Remember that S_3 works by running all models, one per worker, at the same time, and execution is terminated when a model classification other than IDK. This means that the M_A worker will wait when it returns IDK, rather than the M_A worker working on the next input. This is made evident by the very high idle times of all M_A models for $S_{31} - S_{34}$ models as seen in table II-C. Furthermore, by having all the models run on a single input it may be the case that M_A was sufficient to classify the input, but we wasted computation time using M_C when it was not needed. This is a huge problem as even under good conditions where 91.86% of inputs can be resolved by M_A the FPS does not even get above 30.

V. CONCLUSION

In this paper we presented a novel multi-processor IDK cascade that is able to outperform other models when it comes

to classifications per second or FPS. Our scheme involves running each model on a different processor while saving inputs that could not be classified in a doubly linked list for later execution at a later time. We show this is the case by running various tests across various different configurations to show that the model is able to outperform these previous models. We analyze the reason why our model has superior throughput at a small cost of accuracy and discuss the shortcomings of the other models. This small experiment shows the potential of this method as a means to improve multi-processor IDK cascades despite much future work needing to be done.

ACKNOWLEDGMENTS

ChatGPT was used in writing algorithms for different IDK cascade configurations, re-implementing the work from "Work-in-progress: Optimizing IDK cascades skip decisions utilizing random forest predictions" [3], and re-implementing Section 7 from "Scheduling idk classifiers with arbitrary dependencies to minimize the expected time to successful classification" [2]. All Generative-AI output was reviewed, verified, and edited by the authors, who remain solely responsible for all contents.

REFERENCES

- [1] X. Wang, Y. Luo, D. Crankshaw, A. Tumanov, F. Yu, and J. E. Gonzalez, "Idk cascades: Fast deep learning by learning not to overthink," *arXiv preprint arXiv:1706.00885*, 2017.
- [2] T. Abdelzaher, K. Agrawal, S. Baruah, A. Burns, R. I. Davis, Z. Guo, and Y. Hu, "Scheduling idk classifiers with arbitrary dependencies to minimize the expected time to successful classification," *Real-Time Systems*, vol. 3, March 2023.
- [3] R. Katikaneni, A. M. K. Cheng, and T. Carroll, "Work-in-progress: Optimizing idk cascades skip decisions utilizing random forest predictions," 2026, provided source material.
- [4] A.-V. Nguyen, A. M. K. Cheng, and T. Carroll, "Work-in-progress: Utilizing probabilistic analysis to fine-tune optimal idk cascades," in *2024 IEEE Real-Time Systems Symposium (RTSS)*, 2024, pp. 439–442.
- [5] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [6] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, "Do imagenet classifiers generalize to imagenet?" 2019. [Online]. Available: <https://arxiv.org/abs/1902.10811>
- [7] "Metal performance shaders," accessed: September 9, 2026. [Online]. Available: <https://developer.apple.com/documentation/metalperformanceshaders>
- [8] I. J. Ratul, Z. Guo, and K. Yang, "Accuracy-aware idk cascades for real-time object classification at the edge," *Journal of Systems Architecture*, p. 103753, 2026.